Database Project

Due Sunday, 11/08, 11:59 PM

▶ ON THIS PAGE

Design a database for a business that does not have one yet, build it, and use it to answer that business's questions. The project has two parts. Part A is the design and the build: an entity-relationship diagram, tables with declared keys, and data loaded into them. Part B is the answers: business questions asked of the database you built, and how you know the numbers are right.

Choose one scenario below, A, B, C or D, or a domain of your own that meets the bar under E. Each scenario describes what a business keeps today, in spreadsheets and on paper, and the rules it works by. Deciding which tables that becomes, and what the keys are, is your work.

If you did Mini Project 1, you can stay in the same kind of business. Its oat milk orders fit scenario C, the café, and its portfolio rebalance fits scenario B, the investment club. Any scenario is open to you either way.

	DUE	WHAT YOU SUBMIT
Part A	Sunday 11/08, 11:59 PM	The link to `projects/db/`
Part B	Sunday 11/15, 11:59 PM	The link to `projects/db/questions.py`

TL;DR

- **Design a database for a business, build it in DuckDB, and answer the business's questions with SQL.** Choose scenario A, B, C or D below, or a domain of your own under E.
- **This week:** read this whole page, Part A and Part B, and choose a scenario.
- **Part A is due Sunday 11/08:** `projects/db/README.md`, with the design; `projects/db/make_data.py` and the CSV files it writes in

`projects/db/data/`; and `projects/db/build.py`, which builds the database from those files.

- **Part B is due Sunday 11/15:** `projects/db/questions.py`, with the seven questions on your scenario's list, one question of your own, how you know the numbers are right, and the other sections under Part B.
- **I will grade Parts A and B together after Sunday 11/15.** On Sunday 11/08 I will check that you have started: Part A counts as late only if `projects/db/` is still empty that day.
- **Commit before you build and before you answer:** `db: add the question and the plan` before `build.py`, and `db: prediction` before your first answer.
- **Do not commit** `database.duckdb`**.** `build.py` and your CSV files make it again.
- **Going further is required in Part B**, in a direction you choose.

How to Start

1. **Make the folder.** In the VS Code Explorer, click the empty space below the list of files, so that nothing is selected, then click New Folder and type `projects/db`. The `projects` folder has to sit next to the `notebooks` folder, not inside it. Make `projects/db/data` the same way.
2. **Make the README.** Create `projects/db/README.md` and give it the sections under "Part A · The README" as its headings.
3. **Design it, then have your agent draw it.** Work out the tables, keys and relationships, on paper or with your agent. Then fill in README sections 1 to 3: the scenario; the ERD, which your agent writes as mermaid and you read line by line against your design; and one sentence for every table, in your own words, saying what one row of it means. Commit those three sections, with the heading of section 4 still empty, as `db: add the question and the plan`. If you chose A, B, C or D, your scenario is the question.
4. **Then work with your agent.** In a Terminal opened in the root folder of your repository, run `uv add duckdb`, then `uv run marimo edit projects/db/build.py`. When your agent writes a line you could not have written yourself, ask it to explain that line before you keep it. Commit as you go.

ROLE	WHERE IT APPLIES
By hand	What one row of each table means, the design choices in README section 4, and Part B's prediction

ROLE	WHERE IT APPLIES
AI writes, you check	The mermaid ERD, the `CREATE TABLE` statements, the generated data and the queries, each read and tested before you keep it
You direct, you verify	The design, if you work it out with your agent, and the Going further part
You choose the question	Your own business question in Part B, and your own domain if you choose E

A · BabsonAI Labs

BabsonAI Labs is a startup founded by Babson students that builds AI tools for small businesses. It has about 25 people and tracks its work in spreadsheets and chat threads. The founders want to see who works on what, where the money goes, and who is overloaded.

What it keeps today

- **People:** first name, last name, work email, title, start date, and the person they report to. The two founders report to nobody. Work emails change when somebody changes title.
- **Projects:** name, client name (internal projects have none), category (LLM app, data pipeline, internal tool), budget, start date, and status: planned, active, paused or done.
- **Who works on which project:** a person can work on several projects, and a project has several people. For each pairing, the person's role on that project, the hours a week they commit to it, and the date they joined it.
- **Tasks:** each belongs to one project. A description, the date it was created, the person it is assigned to (a new task may be unassigned), the due date, and the date it was completed, which stays empty until it is.
- **Expenses:** each belongs to one project. The date, the amount, the category (GPU cloud, data labeling, software, travel) and a short note.

The rules it works by

- Every person reports to at most one other person, who also works here.
- A budget is more than zero, and so is every expense.
- A task belongs to exactly one project and is assigned to at most one person. It cannot be completed before it was created.
- Nobody commits more than 40 hours a week to a single project, and every commitment is at least one hour.

- A project's status is one of the four listed above.
- A task is assigned only to somebody who works on its project, and never before that person's start date.

The data you need: about 25 people, at least 12 projects, at least 250 tasks and at least 150 expenses.

Business questions

1. Each project's expenses against its budget, and the share of the budget spent, the largest share first.
2. The tasks past their due date and not completed, with the project's name and the name of the person assigned. *(three tables or more)*
3. Each person's hours a week across all active projects, and the people above 40.
4. Expenses by category over the last 90 days, keeping only the categories whose total is above the average of the category totals.
5. The projects with no expenses at all.
6. One row per project with its number of tasks and its total expenses, each checked against counting it on its own. *(three tables or more)*
7. Every task checked against the rule that its assignee works on its project, and a list of any that break it. *(three tables or more) (checks a rule)*

You may need to load the people one row at a time. In `people`, the foreign key for the person somebody reports to points at `people` itself. Loading all the people in one statement, the way notebook 6 loads its towns with `INSERT INTO towns SELECT * FROM town_rows`, fails in whatever order the rows come, because DuckDB checks every row against the table as it was before the statement. One `INSERT` per person, each manager before the people who report to them, works.

B · Babson Investment Club

The Babson Investment Club runs paper portfolios for its members to learn with. Its records are a shared spreadsheet and screenshots from a broker. The club wants a clear history of every trade and to see what each portfolio holds and what it is worth.

What it keeps today

- **Members:** first name, last name, school email, class year and join date.
- **Portfolios:** each is owned by one member, and a member may own several portfolios or none. A name, the date it was created, and a strategy label, which is optional.

- **Assets the club follows:** ticker, name, type (stock, ETF or crypto) and sector, which crypto does not have. A ticker can change while the asset stays the same: Facebook became `META` in 2022, and one asset in your data changes its ticker during the year it covers.
- **Trades:** a portfolio buys or sells an asset. The trade date, whether it was a buy or a sell, the quantity, the price per unit, and a fee, which is zero unless one was charged. A trade's value is its quantity times its price, and the fee is kept apart from it.
- **Closing prices:** for every asset the club follows, one closing price for each trading day. The club keeps the whole history and never writes over an earlier day's price.

The rules it works by

- A trade belongs to exactly one portfolio and one asset.
- A portfolio trades an asset at most once a day.
- A quantity and a price are more than zero, and a fee is never negative.
- A trade is a buy or a sell, and nothing else.
- An asset has at most one closing price on any day.
- A portfolio belongs to exactly one member.
- A trade happens on a day its asset has a closing price.
- A portfolio never sells more units of an asset than it holds at the time. A `CHECK` cannot hold a rule that adds up earlier rows, so this one is checked with a query.

The data you need: about 15 members, at least 12 assets, at least 300 trades, and a closing price for every trading day of the year: every weekday for stocks and ETFs, every day for crypto.

Business questions

1. For each portfolio over the last 30 days: the number of trades, the money spent on buys with fees added, and the money received from sells with fees taken off.
2. One row for each member, members with no portfolio included: their number of portfolios and the total value they bought across them. *(three tables or more)*
3. What each portfolio holds on your `as_of` date: for each asset, the units bought minus the units sold, keeping only those above zero. *(three tables or more)*
4. For every trade in the last 90 days, the closing price on the trade date, and how far the trade price was from it, as a percent of the closing price. *(three*

tables or more)

5. The assets the club follows that no portfolio has ever traded.
6. The asset that changed its ticker: its value traded over the year, under both tickers together.
7. Every trade checked against the rule that it happens on a day its asset has a closing price, and a list of any that break it. *(checks a rule)*

C · Babson Bean Café

Babson Bean is a café on campus. Its till records sales and its loyalty program lives on punch cards, and neither can answer a question. The café wants to know what sells, which add-ons earn their keep, and who its regular customers are.

What it keeps today

- **Loyalty members:** first name, mobile number, email (optional), join date and points balance. A member can change their mobile number. Customers who are not members buy too, and their orders name no customer.
- **The menu:** product name, category (coffee, tea, food, other), price, cost, and whether it is available today.
- **Options a drink can carry:** extra shot, oat milk, vanilla syrup and the like, each with a price, which can be zero, and a cost.
- **Orders:** the date and time, the member if there is one, and the channel: counter or app.
- **Order lines:** an order has one line or more. Each line is one product, with a quantity and the price charged that day, because a menu price can change later.
- **Options on a line:** a line can carry several options, and an option appears on many lines. For each, the price charged that day.
- **Payments:** an order can be paid in more than one payment, part by card and part with points. Each has a method (cash, card, app or points) and an amount in dollars, points included.

The café's revenue is what its lines and their options were charged, however the order was paid.

The rules it works by

- A quantity is more than zero, and so is a payment.
- On the menu, a product's price is never below its cost.
- An order line belongs to exactly one order and one product, and an option on a line belongs to exactly one line.

- An option appears at most once on a line, and it is charged once for every unit on the line: two lattes with oat milk pay for oat milk twice.
- A payment's method is one of the four listed above.
- An order's payments add up to the order's total, its lines and their options together. A `CHECK` cannot hold a rule that spans tables, so this one is checked with a query.

The data you need: at least 40 members, 20 products, eight options and 1,000 orders.

Business questions

1. The three best-selling products last month, by units.
2. Revenue, cost and margin by menu category, with what the options earned counted in, each option under the category of the product on its line. *(three tables or more)*
3. For each member: the number of orders, the total spent and the average order, keeping the members whose total spent is above the members' average. *(three tables or more)*
4. Every order's payments checked against its total, and a list of the orders where they do not match. *(three tables or more) (checks a rule)*
5. The members who joined and have never ordered.
6. The share of orders paid at least partly with points, for each month.
7. The products, available today or not, that have not sold in the last 60 days.

You may need to total the lines and the payments of an order separately before you compare them. Joined together, every line meets every payment.

D · Babson PT Clinic

A physical therapy clinic run by Babson alumni schedules on a paper book and keeps paper charts. It wants to book patients without double-booking a therapist, see what each treatment earns, and find patients it has lost touch with.

What it keeps today

- **Therapists:** first name, last name, credential (DPT or PTA), email and hire date.
- **Patients:** first name, last name, date of birth, phone, email, and an insurance member number, which patients paying for themselves do not have. Two patients can share a name and a birthday.
- **Referrals:** a physician refers a patient: the physician's name, the date and the reason. A patient can have several referrals, or none if they walked in.

- **Treatments the clinic offers:** name, standard minutes, the price it charges and its internal cost.
- **Appointments:** one patient with one therapist, a start date and time, a length of 30, 45 or 60 minutes, and a status: booked, completed or no-show.
- **Treatments given at an appointment:** a completed appointment includes one treatment or more, and a treatment is given at many appointments. For each, the price charged that day and the minutes spent.

The rules it works by

- A therapist never has two appointments starting at the same time.
- An appointment lasts 30, 45 or 60 minutes, and its status is one of the three listed above.
- An appointment belongs to exactly one patient and one therapist.
- A price charged is more than zero, and a cost is never negative.
- An appointment after your `as_of` date is booked; one on or before it is completed or a no-show. The treatments given at an appointment take no more minutes than its length.
- A therapist's appointments never overlap. A constraint holds the same start time; an overlap such as 9:00 for 60 minutes against 9:30 is checked with a query.

The data you need: at least six therapists, 300 patients and 3,000 appointments, including appointments booked for the weeks after your `as_of` date.

Business questions

1. Each therapist's completed appointments last month, and the revenue from them. *(three tables or more)*
2. The no-show rate for each therapist and for each day of the week. The rate is the number of no-shows divided by the number of completed appointments and no-shows together.
3. The patients whose last completed appointment was more than 90 days ago and who have nothing booked, with a phone number, for an outreach call.
4. Revenue, cost and margin for each treatment, the treatments never given included.
5. The patients who were referred and have no appointment at all.
6. One row per patient with their number of referrals and their number of appointments, each checked against counting it on its own. *(three tables or more)*
7. Every completed appointment checked against the rule that the treatments given at it take no more minutes than its length, and a list of any that break it.

(checks a rule)

E · Your Own Domain

Choose a business you know, about as large as the scenarios. It needs no approval. Before your `db: add the question and the plan` commit, write its description in README section 1 in the same shape as A to D: what it keeps today, the rules it works by, and seven business questions. It has to have:

- At least four kinds of thing it keeps, each needing a key, and one where you choose between a key the business already uses and one you invent
- At least two relationships where one row of a table owns many rows of another
- At least one relationship where rows of two tables pair up many to many, with facts of its own about each pairing, such as a quantity, a price or a date
- A column of money and a column of dates
- At least two rules that a column can hold, such as an amount above zero or a status from a fixed list
- At least one relationship that is optional, so that a row with nothing attached to it is a real answer
- At least two of the seven questions using three tables or more
- One rule that takes more than one row to judge, such as payments that add up to an order's total or bookings that never overlap, so that no constraint can hold it, and a question marked *(checks a rule)* that checks it

Part A · The README

`projects/db/README.md`, with these headings, in this order.

1. **The Scenario.** Which one you chose, A to D, or your own domain written out as described under E.

2. **The ERD.** Every table, its columns, its primary key and its foreign keys, and every relationship with both of its ends marked, as a mermaid `erDiagram`. Put it between a line `` ```mermaid `` and a line `` ``` ``, as below, and VS Code's preview and GitHub both show it as a diagram. Chapter A of the SQL handbook, §5, shows how to read one.

````
```mermaid
erDiagram
  CUSTOMERS ||--o{ ORDERS : places
  CUSTOMERS {
    string customer_id PK
````
````

````
    string company_name
  }
  ORDERS {
    int order_id PK
    string customer_id FK
  }
```
````

3. **What One Row Means.** One sentence for each table, in the way Northwind's would read: "One row of `orders` is one order placed by one customer."

4. **Design Choices.** Four short answers, in your own words. Which key you chose where the business already had one, such as an email or a ticker, and why. Which relationships are optional, and what in the scenario tells you so. Which of the scenario's rules your tables cannot hold, and how you will check it instead. And one thing your agent proposed for the design that you changed or did not take, and why; if you designed it alone, one thing you checked in the ERD your agent drew.

Part A · The Build

`projects/db/build.py`, a marimo notebook that builds your database, the file `projects/db/database.duckdb`, from the files in `projects/db/data/`. It has to build it from nothing every time it runs, so the database always matches your CSV files and your `CREATE TABLE` statements.

Nothing may keep the database open. The cell that builds it has to delete the old file first, then open a new one with `with duckdb.connect(database) as _con:` and create the tables and load the data inside that block, so that the file closes when the block ends. Every other cell must read the database through one small function, defined once near the top of each notebook:

```
database = mo.notebook_dir() / "database.duckdb"

def ask(sql):
    with duckdb.connect(database, read_only=True) as _con:
        return _con.sql(sql).pl()
```

`ask("SELECT count(*) FROM orders")` opens the file, runs the query, hands back the result as a table, and closes the file again. So you can rebuild while `questions.py` is open. Close any other program that has the file open, such as a
```

database viewer, before you rebuild. The underscore in `_con` keeps that name inside its cell. marimo needs that, because several cells use the name.

Use these headings, in this order.

1. **The Tables.** One `CREATE TABLE` for each table in your ERD, with every primary key and foreign key declared, and with `NOT NULL`, `CHECK`, `UNIQUE` and `DEFAULT` where the scenario's rules call for them. Chapter D of the SQL handbook, §24, has the syntax.

2. **The Data.** Ask your agent for a script, `projects/db/make_data.py`, that writes data for every table as CSV files in `projects/db/data/`, and load them. The data has to obey every rule, with one exception: for each question marked *(checks a rule)* on your list, break its rule on purpose in exactly two rows, and write down which two in this section. If your tables make the database refuse those rows, your design holds that rule with a constraint: say so here, and show the refusal in that question's answer instead. It has to cover the 12 months up to a date you choose, your `as_of` date, which Part B uses. You can start from this prompt. Replace each part in angle brackets, brackets included, with your own text:

   ```
   Here are the CREATE TABLE statements for my database: <your CR
   Write a Python script, projects/db/make_data.py, that writes o
   per table into projects/db/data/.
   - Follow every rule of my scenario: <your scenario's rules>.
   - Break these rules on purpose, each in exactly two rows, and
     <the rules your checking questions are about>.
   - Make <your scenario's amounts of data>, covering the 12 mont
   - Include on purpose some rows with nothing attached, such as
   - Use a fixed random seed, so that the script writes the same
   ```

   Your scenario says how much data it needs; for your own domain, at least 10 rows in each table that lists things the business keeps and 1,000 rows across the tables that record what happens. Include on purpose some rows with nothing attached, so that the questions about them have an answer. Read the data before you keep it, because an agent's first try usually breaks a rule nobody wrote down: a task assigned before somebody started, a sale of shares nobody held, two appointments at once, a price nobody would charge.

3. **A Look at the Data.** Read one of your CSV files with polars and chart one column whose shape you can judge, such as appointments by day of the week, orders by hour, or one asset's closing prices. Write one sentence on

whether a real business would look like that, and if it would not, what you changed in `make_data.py`.

4. **Row Counts.** Every table with its number of rows.

5. **A Row It Refuses.** Insert one row that breaks one of your rules, and show the message the database gives back. `ask()` only reads, so open a short write connection of its own for the insert, `with duckdb.connect(database) as _con:`, the way notebook 6 does, and put it inside `try` and `except`, so that the rest of the notebook still runs.

6. **Answers You Already Knew.** Three queries whose answers you knew before you ran them, because you know what you asked your agent for, with the answer you expected written next to each.

7. **A View.** One question the business would ask every week, saved with `CREATE VIEW` in the cell that builds the database, so that anybody can ask for it by name and always get the current rows. Notebook 6 covers views in section 5, after `GROUP BY`. At least one of your answers in Part B must use it. Chapter D of the SQL handbook, §28.

Do not commit `database.duckdb`. It is your database, and it stays on your computer; `build.py` and your CSV files are how anybody, me included, makes the same database again. Check that `.gitignore` in the root folder of your repository has the line `*.duckdb*`, which notebook 6 asked for, and add it if it does not. Commit `build.py`, the CSV files, and the script that made them if there is one.

Part B · The Questions

`projects/db/questions.py`, a marimo notebook that has to ask `projects/db/database.duckdb` through an `ask` function of its own, a copy of the one in `build.py`.

Commit your prediction first. Before you answer any question, pick the question for section 3 below, Reading It Back. It has to use three tables or more. Write your prediction in that section and commit it with the message `db: prediction`. Keep the section where it is, after your answers.

If Part B shows a mistake in Part A, fix it in `build.py` or in the data, rebuild, and say what you changed in section 2 below.

A question marked *(checks a rule)* has to list the rows that break its rule. Your data breaks it on purpose in two rows, which you wrote down under The Data in

Part A, so your answer has to find those two and name them. If your tables refuse such rows, show that refusal in the answer instead.

Some questions need SQL the sessions do not cover. Dates, such as "last month" or "each day of the week", are in chapter B, §12. A question that compares rows with an average across them, such as "above the average", needs a query inside another, in chapter C, §21.

One date for every question. Put your `as_of` date in one cell at the top, and count every "last 30 days" back from it: the 30 days ending on that date. "Last month" is the last full calendar month before it.

Answer eight questions: all seven on your scenario's list and one of your own. Your own question has to be one a manager of that business would ask, and it cannot repeat the logic of one on the list. For each question:

- The question as a heading, in words somebody outside this course would understand
- The query, and its result
- One sentence that answers the question with the numbers in it. Where a question can be read two ways, choose one and name your choice in the sentence

Then these sections, in this order.

1. **How I Know These Numbers Are Right.** Reach at least two of your answers a second way, in this notebook, and show that the two agree: a total against the sum of its parts, a count against counting the same rows another way, or a number against the row counts and the data you generated in Part A.
2. **Working With the Agent.** *Pick one piece of AI output you did not accept as-is. What did it give you, what did you change, and how did you know? Point to the commit or the cell.* If the agent got it right the first time: *What did you do to verify that?*
3. **Reading It Back.** The question you picked before answering any, with the prediction you committed then as `db: prediction`: how many rows you expect its joins to produce before any grouping, and why. Then run the joins alone, count the rows, and write whether your prediction matched and why. Then write, in your own words, what one row of the final result means.
4. **Going Further.** What you tried past the main task, what you found, and where it is.
5. **Time Spent.** About how many hours the whole project took you outside class, as one number. It is not graded.

Going Further

- **It is part of Part B, it is required, and any direction counts.** Take at least one step past the eight questions. It does not have to work. Be ambitious here, and use your agent as much as you like. Describe what you tried in Part B's section 4.
- **Directions any scenario can take:**
  - **Put the answer on a chart.** One of your answers as a chart with both axes titled, in the notebook.
  - **Let somebody ask without editing code.** A control in the notebook, such as a dropdown of therapists or portfolios, that changes what a query returns.
  - **Ask it in plain English.** A small tool in your notebook: a question typed in words goes to a model through the API, with your Babson key read from `.env` and your `CREATE TABLE` statements, and the SQL that comes back runs through `ask()`. Compare its answer with yours on one of your questions. This one reaches past what the course covers by then; chapter D of the Python handbook, §32 and §33, has the pieces.
  - **Use real data.** Replace the generated data in one table with real data, such as real closing prices for scenario B, and see which of your answers change.

Submit

Part A, by Sunday 11/08: the link to the folder, in this form.

```
https://github.com/<your-github-username>/<course-code>/tree/main/projects/db
```

Part B, by Sunday 11/15: the link to the notebook, in this form.

```
https://github.com/<your-github-username>/<course-code>/blob/main/projects/db
```

Both links must point into `projects/db/` in your course repository. If your files are anywhere else, move them there before you copy the link.

Copy each link from your browser's address bar, with the folder or the notebook open on github.com. The address under the green **Code** button ends in `.git` and is the wrong one.

Grading

Part A

WHAT I LOOK AT	POINTS
The ERD shows every table, key and relationship, and each relationship's ends match the scenario's rules	3
The tables declare every key, and every rule a column can hold	3
The data loads and obeys the rules, apart from the rows that break one on purpose, and the look at the data, the row counts, the refused row and the view are shown	2
The design choices, in your own words, explain the key you chose, the optional relationships, the rule your tables cannot hold and how you check it, and one proposal of your agent's you changed or one thing you checked	1
The README was committed before `build.py`, and each of the three known answers is compared with what you expected	1

Part B

WHAT I LOOK AT	POINTS
Eight questions answered, each with a query, its result and a sentence with the numbers, and the results are right	4
Two answers reached a second way, and each pair agrees	2
The prediction was committed before the joins were run, and one row of the result is explained	2
Section 2 points to a commit or a cell	1
Section 4 describes a Going further you tried	1

The tables above are a guide. I read each project as a whole.

Last updated: Tuesday, 11/3/2026

For AI agents: this course has a read-only MCP server at `https://oim-mcp.zhili.dev/oim7510/mcp`. Setup notes:

/llms.txt.